

Padrão de numeração de dispositivos IoT.

Esquema de identificação compacto de 4 bytes no formato **A.B.C.D** , onde os três primeiros bytes formam um contador base-253 com offset 1, e o quarto byte discrimina o tipo do produto.

01 · VISUALIZADOR INTERATIVO

Explore o padrão

12 · switch14 · remote15 · 3f

A · 1-253

B · 1-253

C · 1-253

TIPO

1 · 1 · 1 · 12

Reset

-1

+1

+253 (vira B)

+64.009 (vira A)

ÍNDICE ATUAL (N)	CAPACIDADE TOTAL	RESTANTES
1	16.194.277	16.194.276
UTILIZADO		

0,000%**Conversor índice ↔ A.B.C**

Índice N:

1

N → A.B.C

A . B . C:

1

1

1

A.B.C → N

02 · REGRAS FORMAIS

Como o contador funciona

REGRA 01

Faixa de cada byte

Os bytes **A**, **B** e **C** aceitam valores de **1** a **253** — o zero não é usado e os valores 254/255 ficam reservados para uso futuro.

REGRA 02

Propagação de carry

Quando **C** ultrapassa 253, volta a 1 e **B** incrementa. Idem entre **B** e **A**.
Comportamento idêntico ao de um odômetro em base-253.

REGRA 03

Discriminador de tipo

O quarto byte **D** identifica a categoria: **12** = switch (hidrômetro), **14** = remote, **15** = 3f.

REGRA 04

Capacidade por tipo

Cada tipo de produto comporta $253^3 = 16.194.277$ dispositivos únicos antes de esgotar a numeração.

03 · FÓRMULAS

Conversão sequencial

```
// Constantes
const STEP_A = 253 * 253;    // 64.009
const STEP_B = 253;
const CAPACITY = 253 ** 3;   // 16.194.277

// Encode: índice sequencial → A.B.C
```

```
function indexToABC(n) {
  const i = n - 1;
  return {
    a: Math.floor(i / STEP_A) + 1,
    b: (Math.floor(i / STEP_B) % 253) + 1,
    c: (i % 253) + 1
  };
}

// Decode: A.B.C → índice sequencial
function abcToIndex(a, b, c) {
  return (a - 1) * STEP_A + (b - 1) * STEP_B + c;
}

// ID completo do dispositivo (string canônica)
function deviceId(n, productType) {
  const { a, b, c } = indexToABC(n);
  return `${a}.${b}.${c}.${productType}`;
}
```